

Lab 4: Limited Dependent Variable Models

SOCIOL 723, Social Statistics II, Thursday, September 17

Wenhao Jiang

Table of contents

1	Goals	2
2	Binary outcomes	3
2.1	Three scales, one coefficient	3
2.2	Start with predicted probabilities	3
2.3	Then marginal effects: one number per variable	5
2.4	Odds ratios, at most as a footnote	6
2.5	Uncertainty: robust and clustered standard errors	6
2.6	Probit: another S-curve	7
3	Interactions	8
4	Separation	9
5	Outcomes with several categories	10
5.1	Multinomial logit: a nominal outcome	10
5.2	Ordered logit: an ordered outcome	11
5.3	Checking proportional odds	12
6	Count outcomes	13
6.1	Is it overdispersed?	14
6.2	Excess zeros, and the hurdle model	15
6.3	Comparing the count models by AIC and BIC	16
6.4	Does the model reproduce the data?	17
7	Model fit	18
8	Multilevel models	19
8.1	Predictors at both levels	20
8.2	The two variances and the intraclass correlation	20
8.3	Is u_j uncorrelated with SES?	21
8.4	A random-intercept logit	21

9 Event history	22
9.1 A duration with censoring, by construction	22
9.2 Why the naive fixes fail	22
9.3 Kaplan-Meier	23
9.4 The Cox model recovers the truth	23
9.5 Discrete time: the person-period logit	24
10 Exercises	24
11 Session information	25

1 Goals

1. Fit a logit, then read it on the probability scale: predicted probabilities first, average marginal effects second, odds ratios as a footnote.
2. Handle an interaction in a nonlinear model: when to include the product, and how to report the interaction.
3. Fit a multinomial logit on a nominal outcome and an ordered logit on an ordered one; check proportional odds.
4. Fit count models: Poisson, the three overdispersion repairs, the hurdle model; compare them by AIC and BIC.
5. Fit a random-intercept model, read the two variances and the intraclass correlation, and compare it with fixed effects.
6. Handle censored durations: Kaplan-Meier, the Cox model, and the person-period logit.

```
library(lme4)      # lmer, glmer
library(dplyr)
library(ggplot2)
library(marginaleffects)
library(sandwich); library(lmtest)
library(MASS)      # glm.nb, polr
library(AER)       # dispersiontest
library(nnet)      # multinom
library(pscl)      # hurdle, zeroinfl
library(brglm2)    # Firth penalized likelihood
library(pROC)
library(survival)  # Surv, survfit, coxph, survSplit

gss <- readRDS("../Data/gss_earnings.rds")
```

2 Binary outcomes

Outcome: `ba`, whether the respondent holds a bachelor's degree or more. The model is Tuesday's: parents' education, vocabulary score, female, black.

```
f <- ba ~ pareduc + wordsum + female + black

m_logit <- glm(f, data = gss, family = binomial(link = "logit"))
m_probit <- glm(f, data = gss, family = binomial(link = "probit"))
m_lpm <- lm(f, data = gss)

## the logit table from the slides: coefficient, SE, z, exp(coefficient)
round(cbind(coef(summary(m_logit))[, 1:3], exp_b = exp(coef(m_logit))), 3)
#>               Estimate Std. Error z value exp_b
#> (Intercept)   -5.810      0.245 -23.678 0.003
#> pareduc        0.207      0.013  15.389 1.229
#> wordsum        0.391      0.024  16.281 1.479
#> female         0.279      0.078   3.592 1.321
#> black        -0.114      0.110  -1.042 0.892
```

2.1 Three scales, one coefficient

The `wordsum` coefficient is 0.391 on the log-odds scale, $e^{0.391} = 1.48$ on the odds scale, and *not one number* on the probability scale. The first two are exact and uninterpretable to most readers; the third is what the reader wants and depends on where the person starts.

```
## one person at a time: log-odds, odds, probability for wordsum = 3, 6, 9
## with the other covariates at pareduc = 12, female = 0, black = 0
b <- coef(m_logit)
xb <- b["(Intercept)"] + b["pareduc"] * 12 + b["wordsum"] * c(3, 6, 9)
round(cbind(wordsum = c(3, 6, 9), log_odds = xb, odds = exp(xb),
            prob = plogis(xb)), 3)
#>      wordsum log_odds  odds  prob
#> [1,]      3   -2.158 0.116 0.104
#> [2,]      6   -0.983 0.374 0.272
#> [3,]      9    0.191 1.210 0.548
```

Each step of three words adds the same $3 \times 0.391 = 1.17$ to the log-odds and multiplies the odds by the same $e^{1.17} = 3.23$; the probability moves by a different amount each time.

2.2 Start with predicted probabilities

How likely is a degree if your parents finished high school (12 years) rather than college (16)? The observed-value approach: set `pareduc` to 12 for *everyone*, keep every other covariate at its actual value, average the fitted probabilities; repeat at 16.

```
## avg_predictions() does exactly that loop, with a delta-method interval
avg_predictions(m_logit, variables = list(pareduc = c(8, 12, 16, 20)))
#>
#>   pareduc Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>      8      0.198    0.01279 15.5  <0.001  177.2 0.173  0.223
#>     12      0.343    0.00912 37.6  <0.001 1027.2 0.325  0.361
#>     16      0.521    0.01098 47.5  <0.001    Inf 0.500  0.543
#>     20      0.694    0.01795 38.7  <0.001    Inf 0.659  0.729
#>
#> Type: response
```

```
## the contrast we report: high school to college, 18 points
avg_comparisons(m_logit, variables = list(pareduc = c(12, 16)))
#>
#>   Estimate Std. Error    z Pr(>|z|)      S 2.5 % 97.5 %
#>    0.178    0.0117 15.2  <0.001 170.4 0.155  0.201
#>
#> Term: pareduc
#> Type: response
#> Comparison: 16 - 12
```

```
plot_predictions(m_logit, by = "pareduc") +
  labs(x = "Parents' education (years)", y = "Pr(bachelor's or more)") +
  theme_minimal(base_size = 10)
```

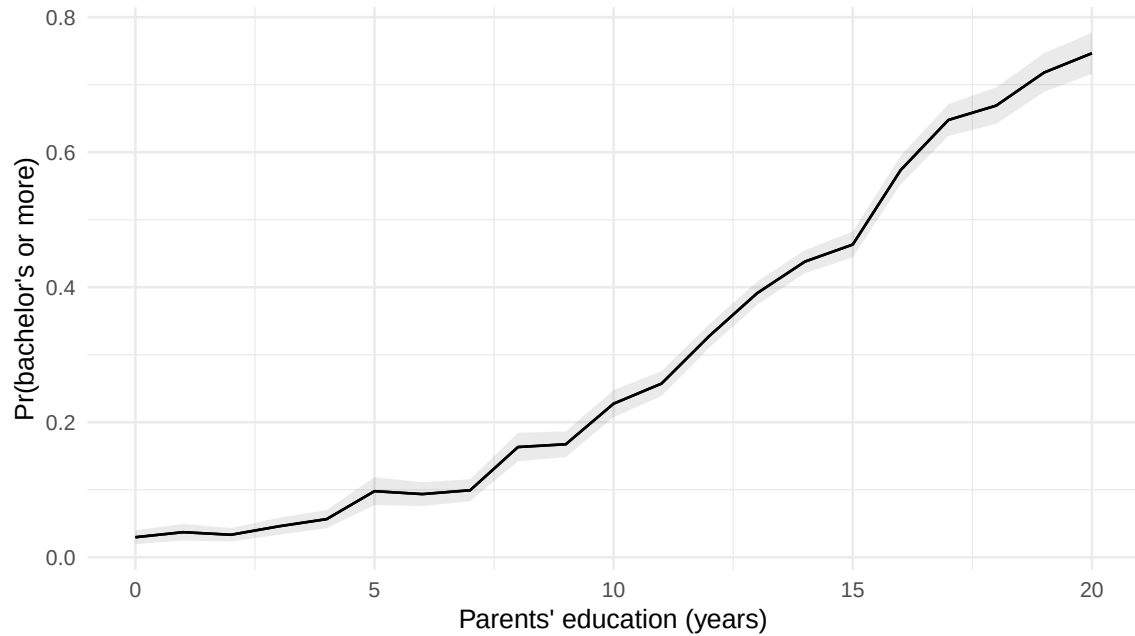


Figure 1: Average predicted probability of holding a BA by parents' education (observed-value approach), with 95% intervals.

Contrast with setting everything else to its mean, which describes one hypothetical person (a 0.5 woman, a 0.13 Black respondent):

```
predictions(m_logit, newdata = datagrid(pareduc = c(12, 16)))
#>
#>   pareduc Estimate Pr(>|z|)      S 2.5 % 97.5 %
#>    12     0.272  <0.001 179.2 0.248 0.297
#>    16     0.461   0.0161   6.0 0.429 0.493
#>
#> Type: invlink(link)
```

2.3 Then marginal effects: one number per variable

```
## AME: each person's own slope dp/dx = beta * p(1 - p), averaged
avg_slopes(m_logit)
#>
#>   Term Contrast Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#> black      1 - 0 -0.0219  0.02097 -1.05  0.296   1.8 -0.0630 0.0192
#> female     1 - 0  0.0536  0.01488  3.60 <0.001  11.6  0.0245 0.0828
#> pareduc    dY/dX  0.0397  0.00226 17.57 <0.001 227.1  0.0353 0.0441
#> wordsum    dY/dX  0.0752  0.00395 19.03 <0.001 265.8  0.0675 0.0830
#>
#> Type: response
```

Compare with the marginal effect at the mean and with the LPM coefficients:

```
mem <- slopes(m_logit, newdata = "mean")
comparison <- data.frame(
  variable = c("pareduc", "wordsum", "female", "black"),
  AME = avg_slopes(m_logit)$estimate,
  MEM = mem$estimate,
  LPM = coef(m_lpm)[-1]
)
round(comparison[, -1], 4) |> `rownames<-`(comparison$variable)
#>           AME      MEM      LPM
#> pareduc -0.0219 -0.0241  0.0348
#> wordsum  0.0536  0.0630  0.0721
#> female   0.0397  0.0446  0.0500
#> black    0.0752  0.0845 -0.0379
```

- The AME and the LPM coefficients are close. This is typical, and it is the main reason people defend the LPM for descriptive and causal work.
- The MEM answers a different question: the slope for one hypothetical person at the mean of everything, including the means of dummies.
- Report the AME (Hanmer and Kalkan 2013).

2.4 Odds ratios, at most as a footnote

```
round(exp(cbind(OR = coef(m_logit), confint.default(m_logit))), 3)
#>           OR 2.5 % 97.5 %
#> (Intercept) 0.003 0.002  0.005
#> pareduc     1.229 1.198  1.262
#> wordsum     1.479 1.411  1.550
#> female      1.321 1.135  1.538
#> black       0.892 0.719  1.106
```

Each correct word multiplies the *odds* of a BA by 1.48. That is one number for everyone, because the model is linear on the log-odds scale. It is not “48% more likely”: the probability change depends on where the person starts, which is why the probabilities came first.

2.5 Uncertainty: robust and clustered standard errors

The sandwich from Week 2 works on `glm` objects, and `marginalEffects` will carry a chosen variance through to the AMEs.

```
round(cbind(
  classical = sqrt(diag(vcov(m_logit))),
  HCO       = sqrt(diag(vcovHC(m_logit, type = "HCO"))),
  cluster    = sqrt(diag(vcovCL(m_logit, cluster = ~ psu)))
)
```

```

), 4)
#>
#>          classical      HCO cluster
#> (Intercept)    0.2454 0.2662  0.2697
#> pareduc       0.0134 0.0155  0.0159
#> wordsum       0.0240 0.0247  0.0246
#> female        0.0776 0.0771  0.0734
#> black         0.1098 0.1098  0.1097

avg_slopes(m_logit, vcov = ~ psu)
#>
#>      Term Contrast Estimate Std. Error      z Pr(>|z|)      S  2.5 % 97.5 %
#> black      1 - 0  -0.0219   0.02095 -1.05   0.295    1.8 -0.0630 0.0191
#> female     1 - 0   0.0536   0.01404  3.82  <0.001   12.9  0.0261 0.0811
#> pareduc    dY/dX   0.0397   0.00266 14.90  <0.001  164.4  0.0345 0.0449
#> wordsum    dY/dX   0.0752   0.00409 18.40  <0.001  248.8  0.0672 0.0832
#>
#> Type: response

```

2.6 Probit: another S-curve

```

## coefficients differ by a scale factor; AMEs do not
round(cbind(logit = coef(m_logit), probit = coef(m_probit),
            ratio = coef(m_logit) / coef(m_probit)), 3)
#>
#>      logit probit ratio
#> (Intercept) -5.810 -3.367 1.726
#> pareduc      0.207  0.117 1.768
#> wordsum      0.391  0.233 1.678
#> female       0.279  0.159 1.751
#> black       -0.114 -0.070 1.643
round(cbind(AME_logit = avg_slopes(m_logit)$estimate,
            AME_probit = avg_slopes(m_probit)$estimate), 4) |>
  `rownames<-`(avg_slopes(m_logit)$term)
#>
#>      AME_logit AME_probit
#> black      -0.0219  -0.0225
#> female      0.0536   0.0515
#> pareduc     0.0397   0.0378
#> wordsum     0.0752   0.0754

```

The coefficients differ by a factor of about 1.6 to 1.7 because the two S-curves have different steepness at the center. The AMEs agree to two decimals. Default to logit; use probit when the next step needs a normal (selection models, bivariate probit) or when the subfield expects it.

! Important

Never compare raw logit or probit coefficients across nested models or across groups. Adding a covariate rescales every coefficient, even ones whose effect is unchanged (Mood 2010). Compare AMEs or predicted probabilities.

3 Interactions

Does parents' education matter more for women? Two questions: should the product `pareduc * female` be in the model, and do the models with and without it agree on the probability scale?

```
## no product: one log-odds slope for pareduc, forced equal for men and women
m_np <- glm(ba ~ pareduc + female + wordsum + black, data = gss, family = binomial)
## with product: the slope is beta_1 + beta_12 * female; the data choose beta_12
m_int <- glm(ba ~ pareduc * female + wordsum + black, data = gss, family = binomial)
round(coef(summary(m_int))[, 1:3], 3)
#>
#> Estimate Std. Error z value
#> (Intercept) -5.675 0.297 -19.082
#> pareduc 0.197 0.018 10.857
#> female -0.012 0.377 -0.031
#> wordsum 0.391 0.024 16.286
#> black -0.116 0.110 -1.055
#> pareduc:female 0.021 0.027 0.786
## LR test of the product
anova(m_np, m_int, test = "LRT")
#> Analysis of Deviance Table
#>
#> Model 1: ba ~ pareduc + female + wordsum + black
#> Model 2: ba ~ pareduc * female + wordsum + black
#> Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1 3504 3959
#> 2 3503 3958 1 0.619 0.43
```

The product is 0.021 on the log-odds scale, $z = 0.79$. That is the wrong scale for the question. The AME of parents' education, by gender, in both models:

```
## no product: the AMEs still differ by gender, through  $p(1 - p)$  alone
avg_slopes(m_np, variables = "pareduc", by = "female")
#>
#> female Estimate Std. Error z Pr(>|z|) S 2.5 % 97.5 %
#> 0 0.0386 0.00220 17.6 <0.001 227.4 0.0343 0.0429
#> 1 0.0409 0.00234 17.4 <0.001 223.8 0.0363 0.0455
#>
#> Term: pareduc
```



```
#> Type: response
#> Comparison: dY/dX
## with product: they differ through  $p(1 - p)$  and through  $\beta_{12}$ 
avg_slopes(m_int, variables = "pareduc", by = "female")
#>
#> female Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#>      0  0.0371    0.00296 12.5   <0.001 116.8 0.0313 0.0429
#>      1  0.0428    0.00335 12.8   <0.001 121.4 0.0362 0.0493
#>
#> Term: pareduc
#> Type: response
#> Comparison: dY/dX
```

```
## the interaction we report: the difference in AMEs, with an interval
avg_slopes(m_int, variables = "pareduc", by = "female",
           hypothesis = "b2 - b1 = 0")
#>
#> Hypothesis Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#>    b2-b1=0  0.00571    0.0044 1.3    0.195 2.4 -0.00292 0.0143
#>
#> Type: response
```

- **Include the product** whenever the question is whether X_1 's effect differs by X_2 . Without it the answer is fixed in advance: zero on the log-odds scale, and on the probability scale whatever the S-curve gives (the no-product model already shows women with a larger AME, because they sit where the curve is steeper: Ai and Norton 2003).
- **The two models agree** on probabilities only when $\hat{\beta}_{12}$ is near zero, as here. When it is not, the no-product model has the wrong probabilities in the cells.
- **Report** the difference in AMEs with its interval, here 0.006 per year, SE 0.004. The answer is no.

4 Separation

Construct a variable that perfectly predicts the outcome, and watch the MLE run off to infinity.

```
gss2 <- gss |> mutate(perfect = as.integer(educ >= 18 & ba == 1))
m_sep <- glm(ba ~ pareduc + perfect, data = gss2, family = binomial)
round(coef(summary(m_sep))[, 1:2], 3)
#>
#> Estimate Std. Error
#> (Intercept) -3.899    0.207
#> pareduc      0.232    0.015
#> perfect      19.320   271.710
```

An enormous coefficient with an enormous standard error is the signature. The fix is Firth's

penalized likelihood:

```
m_firth <- glm(ba ~ pareduc + perfect, data = gss2,
               family = binomial, method = "brglmFit")
round(coef(summary(m_firth))[, 1:2], 3)
#>               Estimate Std. Error
#> (Intercept)   -3.893      0.207
#> pareduc         0.232      0.015
#> perfect         7.803      1.424
```

Finite estimates, usable standard errors, and the variable is retained (Zorn 2005).

5 Outcomes with several categories

5.1 Multinomial logit: a nominal outcome

Where a respondent lives, in four census regions. The categories have no order, so one logit for every region against a base (the Midwest).

```
## collapse the nine GSS regions to four; Midwest first so it is the base
gss$region4 <- factor(case_when(
  gss$region %in% c("New England", "Middle Atlantic") ~ "Northeast",
  gss$region %in% c("E. North Central", "W. North Central") ~ "Midwest",
  gss$region %in% c("South Atlantic", "E. South Central", "W. South Central") ~ "South",
  TRUE ~ "West"), levels = c("Midwest", "Northeast", "South", "West"))
table(gss$region4)
#>
#>   Midwest Northeast      South      West
#>      861      590      1298      760

m_mn <- multinom(region4 ~ educ + black + female + age, data = gss, trace = FALSE)
## one row per non-base region: exp(coef) multiplies the odds AGAINST THE MIDWEST
round(exp(coef(m_mn)), 2)
#>               (Intercept) educ black female  age
#> Northeast             0.21 1.08  1.32   1.08 1.00
#> South                 1.52 0.98  2.88   1.21 1.00
#> West                  0.99 1.01  0.82   1.13 0.99
```

Read the **black** column: being Black multiplies the odds of South-versus-Midwest by about 2.9 and of West-versus-Midwest by about 0.8. Those are odds against the base, not probabilities; the four probabilities move together and add to one:

```
## observed-value predicted probabilities, one column per region
avg_predictions(m_mn, variables = "black")
#>
#>      Group black Estimate Std. Error      z Pr(>|z|)      S  2.5 % 97.5 %
```

```
#> Midwest      0    0.261    0.00806 32.38 <0.001 761.6 0.2451 0.277
#> Midwest      1    0.160    0.01591 10.03 <0.001  76.3 0.1284 0.191
#> Northeast    0    0.173    0.00692 25.03 <0.001 456.9 0.1596 0.187
#> Northeast    1    0.141    0.01517  9.29 <0.001  65.8 0.1112 0.171
#> South        0    0.331    0.00863 38.36 <0.001 1066.8 0.3142 0.348
#> South        1    0.582    0.02132 27.28 <0.001  542.1 0.5398 0.623
#> West         0    0.235    0.00779 30.15 <0.001  661.1 0.2196 0.250
#> West         1    0.118    0.01389  8.49 <0.001  55.4 0.0907 0.145
#>
#> Type: probs
## check: for each person the four probabilities sum to one
range(rowSums(fitted(m_mn)))
#> [1] 1 1

## do the covariates matter at all? LR against the intercept-only model
m_mn0 <- multinom(region4 ~ 1, data = gss, trace = FALSE)
lr <- 2 * (as.numeric(logLik(m_mn)) - as.numeric(logLik(m_mn0)))
df <- length(coef(m_mn)) - length(coef(m_mn0))
c(LR = round(lr, 1), df = df)
#>      LR      df
#> 171.2  12.0
pchisq(lr, df, lower.tail = FALSE) < 0.001
#> [1] TRUE
```

5.2 Ordered logit: an ordered outcome

Highest degree, five ordered categories. One coefficient vector and four cutpoints.

```
m_ord <- polr(degree ~ pareduc + wordsum + female + black,
              data = gss, Hess = TRUE)
round(coef(summary(m_ord)), 4)
#>
#>                Value Std. Error t value
#> pareduc          0.2082     0.0104 19.9296
#> wordsum           0.3578     0.0192 18.5908
#> female            0.3349     0.0646  5.1829
#> black            -0.0238     0.0914 -0.2607
#> < HS|HS          1.8372     0.1616 11.3667
#> HS|Junior college 5.2081     0.1788 29.1209
#> Junior college|Bachelor 5.6724     0.1823 31.1134
#> Bachelor|Graduate 7.3582     0.1973 37.3032
```

`polr()` does not print *p*-values; get them from the *t*-values, or better, work on the probability scale.

```
## the cutpoints are gates: for one person,  $X'\beta$  and the four cumulative
## probabilities  $Pr(Y \leq j) = \text{Lambda}(\text{kappa}_j - X'\beta)$ ; differences are the
## five category probabilities
```

```
b <- coef(m_ord); kappa <- m_ord$zeta
x <- c(pareduc = 12, wordsum = 5, female = 0, black = 0)
xb <- sum(b * x)
cum <- plogis(kappa - xb)
round(c(xb = xb), 3)
#>      xb
#> 4.288
round(rbind(cumulative = c(cum, 1), category = diff(c(0, cum, 1))), 3) |>
  `colnames<-`(levels(gss$degree))
#>           < HS      HS Junior college Bachelor Graduate
#> cumulative 0.079 0.715           0.800      0.956      1.000
#> category   0.079 0.636           0.085      0.156      0.044
```

```
## the same, averaged over everyone, at two vocabulary scores
```

```
avg_predictions(m_ord, variables = list(wordsum = c(3, 9)), type = "probs")
#>
#>           Group wordsum Estimate Std. Error      z Pr(>|z|)      S 2.5 % 97.5 %
#> < HS              3  0.1265   0.00888 14.25  <0.001 150.7 0.1091 0.1439
#> < HS              9  0.0190   0.00195  9.72  <0.001  71.8 0.0152 0.0228
#> HS                3  0.6228   0.01180 52.80  <0.001   Inf 0.5997 0.6459
#> HS                9  0.2839   0.01091 26.02  <0.001 493.4 0.2625 0.3053
#> Junior college    3  0.0719   0.00446 16.12  <0.001 191.8 0.0632 0.0807
#> Junior college    9  0.0936   0.00506 18.49  <0.001 251.2 0.0837 0.1035
#> Bachelor          3  0.1373   0.00812 16.90  <0.001 210.5 0.1214 0.1532
#> Bachelor          9  0.3534   0.00993 35.58  <0.001 918.6 0.3339 0.3729
#> Graduate          3  0.0414   0.00367 11.27  <0.001  95.5 0.0342 0.0486
#> Graduate          9  0.2501   0.01092 22.91  <0.001 383.5 0.2287 0.2715
#>
#> Type: probs
```

5.3 Checking proportional odds

One β for every cutpoint is an assumption: vocabulary pushes people past the high-school line and the bachelor's line by the same log-odds. Check it the direct way, with four separate binary logits, “above cut j ” versus not.

```
## one binary logit per cut; under proportional odds the four rows agree
cuts <- levels(gss$degree)[-5]
above <- sapply(1:4, function(k) {
  yk <- as.integer(as.integer(gss$degree) > k)    # 1 if above cut k
```

```

coef(glm(yk ~ pareduc + wordsum + female + black, data = gss, family = binomial))[-1]
})
colnames(above) <- paste("above", cuts)
round(cbind(above, ordered = coef(m_ord)), 3)
#>           above < HS above HS above Junior college above Bachelor ordered
#> pareduc      0.231    0.198                0.207            0.180    0.208
#> wordsum      0.407    0.348                0.391            0.331    0.358
#> female       0.252    0.367                0.279            0.379    0.335
#> black        0.116   -0.076                -0.114            0.125   -0.024

```

The ordered model's **wordsum** coefficient is a compromise between the four; proportional odds is a fair summary here.

```

## the formal check: a multinomial logit on degree frees all four rows
m_mn_deg <- multinom(degree ~ pareduc + wordsum + female + black, data = gss, trace = FALSE)
k_ord <- length(coef(m_ord)) + length(m_ord$zeta) # 4 slopes + 4 cutpoints
k_mn <- length(coef(m_mn_deg)) # 4 rows x 5 columns
lr <- 2 * (as.numeric(logLik(m_mn_deg)) - as.numeric(logLik(m_ord)))
c(LR = round(lr, 1), df = k_mn - k_ord,
  p = round(pchisq(lr, k_mn - k_ord, lower.tail = FALSE), 3))
#>      LR      df      p
#> 27.300 12.000 0.007

## the same comparison by AIC and BIC
rbind(ordered = c(k = k_ord, AIC = AIC(m_ord), BIC = BIC(m_ord)),
      multinomial = c(k = k_mn, AIC = AIC(m_mn_deg), BIC = BIC(m_mn_deg)))
#>           k  AIC  BIC
#> ordered    8 8628 8677
#> multinomial 20 8625 8748

```

The LR rejects at $n = 3,509$, as everything does (Week 3). BIC charges $12 \log 3509 = 98$ for the twelve extra parameters and gets back 27, so it keeps the ordered model; AIC charges 24 and is close to indifferent. Judge the table first, then the test. If one covariate fails badly, free that covariate only (`ordinal::clm` with a `nominal =` term).

6 Count outcomes

Outcome: **childs**, the number of children.

```

f_c <- childs ~ educ + female + black + exper + evermar
m_pois <- glm(f_c, data = gss, family = poisson(link = "log"))
## exp(coef) multiplies the expected count
round(cbind(coef(summary(m_pois))[, 1:2], exp_b = exp(coef(m_pois))), 3)
#>           Estimate Std. Error exp_b

```

```
#> (Intercept)  -0.006      0.088 0.994
#> educ         -0.045      0.005 0.956
#> female        0.009      0.027 1.009
#> black         0.360      0.035 1.433
#> exper         0.012      0.001 1.012
#> evermar       0.947      0.043 2.578
```

6.1 Is it overdispersed?

The Poisson makes one strong claim: $\text{Var}(Y_i | X_i) = \theta_i$. Check it three ways.

```
## raw: variance against mean
c(mean = mean(gss$chlds), var = var(gss$chlds))
#> mean var
#> 1.604 2.109

## Pearson dispersion: under the Poisson each  $(y - \theta)^2 / \theta$  averages 1
n <- nrow(gss); k <- length(coef(m_pois))
phi <- sum((gss$chlds - fitted(m_pois))^2 / fitted(m_pois)) / (n - k)
c(phi = phi)
#> phi
#> 1.149

## formal test, using only the Poisson fit
dispersiontest(m_pois, trafo = 1)
#>
#> Overdispersion test
#>
#> data: m_pois
#> z = 4, p-value = 0.00004
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#> alpha
#> 0.1676
```

The counts scatter 15% more than the Poisson allows. The claim is false, so the Poisson SE is wrong. Three repairs, each replacing the claim with a different one:

```
m_qp <- glm(f_c, data = gss, family = quasipoisson) # Var = phi * theta
m_nb <- glm.nb(f_c, data = gss) # Var = theta + theta^2 / alpha

round(cbind(
  poisson = sqrt(diag(vcov(m_pois))),
  quasi = sqrt(diag(vcov(m_qp))),
```

```

negbin = sqrt(diag(vcov(m_nb)))[names(coef(m_pois))],
robust = sqrt(diag(vcovHC(m_pois, type = "HC0")) # no claim at all
), 4)
#>          poisson   quasi negbin robust
#> (Intercept)  0.0876 0.0939 0.0900 0.1012
#> educ         0.0048 0.0051 0.0049 0.0052
#> female       0.0269 0.0288 0.0277 0.0278
#> black        0.0349 0.0374 0.0361 0.0373
#> exper        0.0013 0.0014 0.0014 0.0013
#> evermar      0.0430 0.0461 0.0437 0.0551

## the quasi-Poisson SE is the Poisson SE times sqrt(phi); check on evermar
c(by_hand = sqrt(diag(vcov(m_pois)))[ "evermar" ] * sqrt(phi),
  quasi   = sqrt(diag(vcov(m_qp)))[ "evermar" ])
#> by_hand.evermar   quasi.evermar
#>          0.04605          0.04605

```

The coefficients barely move (check `coef(m_nb)` against `coef(m_pois)`); the standard errors do. The robust SE for `evermar` is much larger than the other two because the excess scatter is not uniform: it sits among the never married, exactly the group this dummy compares.

```

## the negative binomial's alpha (theta in MASS): large means close to Poisson
c(alpha = m_nb$theta, se_alpha = m_nb$SE.theta)
#>      alpha se_alpha
#>      31.25   14.91

```

6.2 Excess zeros, and the hurdle model

```

## share of zeros: observed, and averaged over the fitted distributions
c(observed = mean(gss$childs == 0),
  poisson = mean(dpois(0, fitted(m_pois))),
  negbin   = mean(dnbinom(0, mu = fitted(m_nb), size = m_nb$theta)))
#> observed poisson   negbin
#>   0.3007   0.2519   0.2592

```

Thirty percent are childless; neither model has a way to make zeros special. The hurdle model does: a logit for zero versus positive, then a truncated Poisson for the positives, the same covariates in both parts.

```

m_hur <- hurdle(f_c, data = gss, dist = "poisson")
## the two parts tell different stories
round(cbind(any_child = exp(coef(m_hur, model = "zero")),
          how_many    = exp(coef(m_hur, model = "count"))), 2)
#>          any_child how_many

```

```
#> (Intercept)      1.08      2.09
#> educ            0.89      0.96
#> female          1.34      0.93
#> black           2.92      1.25
#> exper           1.03      1.01
#> evermar         8.91      1.35
## the zero share is matched exactly: the logit part is fitted to it
mean(predict(m_hur, type = "prob")[, 1])
#> [1] 0.3007
```

Marriage multiplies the odds of *any* child by about 9 but the count among parents by about 1.35; women are likelier to have any child and have slightly fewer. One Poisson averaged these two processes.

6.3 Comparing the count models by AIC and BIC

Poisson and negative binomial are nested (the Poisson is the negative binomial with $\alpha \rightarrow \infty$), so an LR test applies. The hurdle is not nested in either, so use AIC and BIC, which need only a log-likelihood and a parameter count.

```
## LR: negative binomial against Poisson, one parameter (alpha)
2 * (as.numeric(logLik(m_nb)) - as.numeric(logLik(m_pois)))
#> [1] 4.805

## AIC = -2 loglik + 2k; BIC = -2 loglik + k log n
mods <- list(Poisson = m_pois, `Neg. binomial` = m_nb, Hurdle = m_hur)
tab <- t(sapply(mods, function(m) {
  ll <- logLik(m)
  c(logLik = as.numeric(ll), k = attr(ll, "df"),
    AIC = -2 * as.numeric(ll) + 2 * attr(ll, "df"),
    BIC = -2 * as.numeric(ll) + attr(ll, "df") * log(n))
}))
round(tab, 1)
#>           logLik  k   AIC   BIC
#> Poisson      -5383  6 10778 10815
#> Neg. binomial -5380  7 10775 10818
#> Hurdle       -5228 12 10479 10553
## check against the built-in functions
round(c(AIC(m_hur), BIC(m_hur)), 1)
#> [1] 10479 10553
```

Read the Poisson-versus-hurdle line the Week 3 way. The hurdle adds six parameters (a second intercept and five more coefficients). BIC charges $6 \log 3509 = 49$ for them, AIC charges 12; the log-likelihood rises by about 155, so -2ℓ falls by about 310. Both criteria prefer the hurdle by a

wide margin, and BIC's heavier penalty does not change the verdict. The negative binomial is the interesting line: one extra parameter, a gain of 2.4 in log-likelihood. AIC prefers it to the Poisson by about 3; BIC prefers the Poisson by about 3. That is Week 3's disagreement in the wild: the evidence for α is real (the LR is 4.8) but not strong enough to pay BIC's $\log 3509 = 8.2$ per parameter. Both criteria agree that the zeros, not the variance, are where the Poisson is wrong.

i Note

AIC and BIC compare models fitted to the *same rows* with the *same outcome*. They say nothing about whether either model is right; the predictive check below does.

6.4 Does the model reproduce the data?

```
## for each count k, average Pr(Y = k) over everyone's fitted distribution
ks <- 0:8
dist <- bind_rows(
  tibble::tibble(k = ks, source = "observed",
    p = as.numeric(table(factor(gss$childs, levels = ks))) / n),
  tibble::tibble(k = ks, source = "Poisson",
    p = sapply(ks, function(j) mean(dpois(j, fitted(m_pois))))),
  tibble::tibble(k = ks, source = "Neg. binomial",
    p = sapply(ks, function(j) mean(dnbinom(j, mu = fitted(m_nb), size = m_nb$th
  tibble::tibble(k = ks, source = "Hurdle",
    p = colMeans(predict(m_hur, type = "prob")[, 1:9]))
) |> mutate(source = factor(source, levels = c("observed", "Poisson", "Neg. binomial", "Hurdle"))

ggplot(dist, aes(k, p, fill = source)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("observed" = "grey30", "Poisson" = "navy",
    "Neg. binomial" = "firebrick", "Hurdle" = "darkgreen")) +
  labs(x = "number of children", y = "proportion", fill = NULL) +
  theme_minimal(base_size = 10)
```

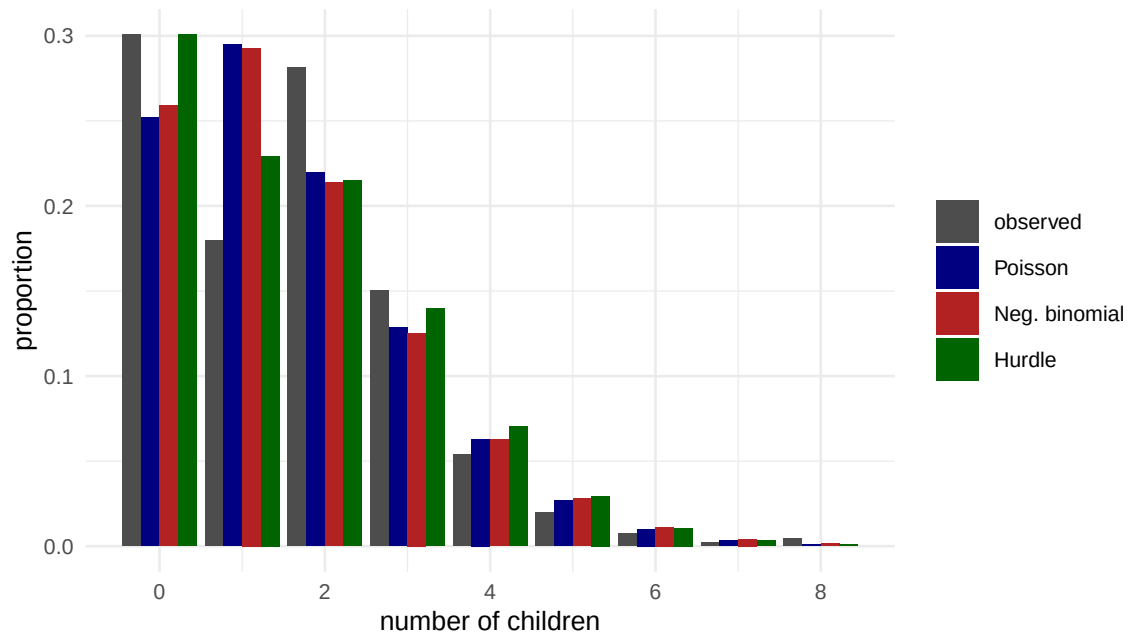


Figure 2: Observed distribution of the number of children against the distributions implied by the Poisson, negative binomial, and hurdle fits.

This predictive check is the single most informative count diagnostic, and the one people most often skip. The hurdle matches the zeros by construction; no count model matches the bump at two children.

i Note

Which count model. Start with Poisson and robust SEs. Run the overdispersion test; if it rejects and you need a likelihood (prediction, AIC, an LR test), fit the negative binomial. Look at the observed and predicted distributions. Choose hurdle or zero-inflated on substance: hurdle when crossing from zero to one is a different *process*, zero-inflated when some zeros are a different *kind* of person. The two are not nested; AIC or BIC between them.

7 Model fit

```
AIC(m_np, m_int)
#>      df  AIC
#> m_np   5 3969
#> m_int  6 3970
BIC(m_np, m_int)
#>      df  BIC
#> m_np   5 4000
```

```
#> m_int 6 4007
```

```
## McFadden's pseudo-R2
```

```
m_null <- glm(ba ~ 1, data = gss, family = binomial)
```

```
1 - as.numeric(logLik(m_logit)) / as.numeric(logLik(m_null))
```

```
#> [1] 0.1669
```

```
roc_obj <- roc(gss$ba, fitted(m_logit), quiet = TRUE)
```

```
plot(roc_obj, col = "navy", lwd = 2, legacy.axes = TRUE)
```

```
text(0.6, 0.2, paste("AUC =", round(auc(roc_obj), 3)))
```

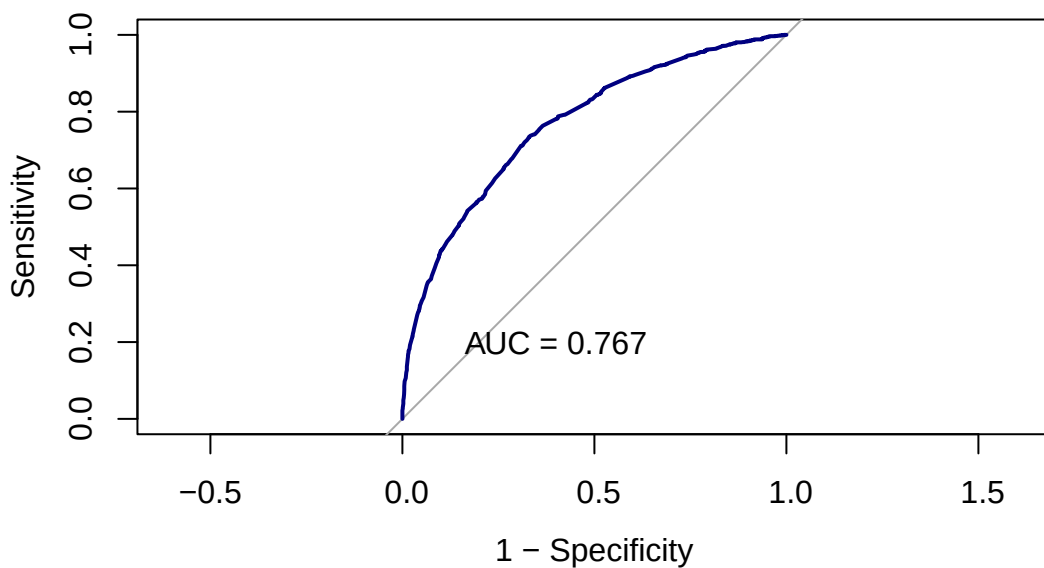


Figure 3: ROC curve for the logit model of holding a bachelor's degree.

```
## classification accuracy against the null rule of "always predict the majority"
```

```
pred <- as.integer(fitted(m_logit) > 0.5)
```

```
c(model = mean(pred == gss$ba),  
  always_majority = max(mean(gss$ba), 1 - mean(gss$ba)))
```

```
#>          model always_majority
```

```
#>          0.7048          0.5893
```

8 Multilevel models

The GSS extract has no natural grouping, so this section uses the High School and Beyond data that ship with `nlme`: 7,185 students in 160 schools, a math achievement score, student SES, and school sector.

```
data(MathAchieve, package = "nlme"); data(MathAchSchool, package = "nlme")
hsb <- merge(as.data.frame(MathAchieve),
            as.data.frame(MathAchSchool)[, c("School", "Sector")], by = "School")
hsb$School <- factor(as.character(hsb$School)) # plain factor, not ordered
c(students = nrow(hsb), schools = length(unique(hsb$School)))
#> students schools
#>      7185      160
range(table(hsb$School))
#> [1] 14 67
```

8.1 Predictors at both levels

```
m_ols <- lm(MathAch ~ SES + Sector, data = hsb)
m_re  <- lmer(MathAch ~ SES + Sector + (1 | School), data = hsb, REML = FALSE)
m_fe  <- lm(MathAch ~ SES + Sector + School, data = hsb)

round(cbind(
  pooled      = coef(m_ols)[c("SES", "SectorCatholic")],
  pooled_clSE = sqrt(diag(vcovCL(m_ols, cluster = hsb$School)))[c("SES", "SectorCatholic")],
  random_int  = fixef(m_re)[c("SES", "SectorCatholic")],
  random_SE   = sqrt(diag(vcov(m_re)))[c("SES", "SectorCatholic")],
  fixed_eff   = coef(m_fe)[c("SES", "SectorCatholic")]), 3)
#>          pooled pooled_clSE random_int random_SE fixed_eff
#> SES          2.949      0.128      2.377      0.105      2.191
#> SectorCatholic 1.935      0.317      2.100      0.339      2.835
```

Sector is NA under fixed effects: it is constant within school, so the school dummies absorb it. The random-intercept model estimates it, with a standard error that treats the 160 schools, not the 7,185 students, as the units. The SES slope sits between the pooled and the fixed-effects one: the random-intercept model uses both within-school and between-school variation.

8.2 The two variances and the intraclass correlation

```
## tau2 = between-school variance of the intercepts, sigma2 = within-school
vc <- as.data.frame(VarCorr(m_re))
tau2 <- vc$vcov[1]; sigma2 <- vc$vcov[2]
c(tau2 = tau2, sigma2 = sigma2, ICC = tau2 / (tau2 + sigma2))
#>      tau2      sigma2      ICC
#> 3.62187 37.03279 0.08909

## is tau2 zero? LR against OLS with the same x, one parameter pinned at zero
2 * (as.numeric(logLik(m_re)) - as.numeric(logLik(m_ols)))
#> [1] 332.2
```

Of the variation SES and sector leave unexplained, 9% is between schools and 91% within. The LR statistic is enormous; schools still differ after the controls.

8.3 Is u_j uncorrelated with SES?

The random-intercept model assumes the school shocks are uncorrelated with the covariates. Check it, then fix it.

```
## do high-SES schools have higher shocks? u_hat: the model's predicted u_j
hsb$meanSES <- ave(hsb$SES, hsb$School)
u_hat <- ranef(m_re)$School[, 1]
school_meanSES <- tapply(hsb$meanSES, hsb$School, mean)[rownames(ranef(m_re)$School)]
round(cor(school_meanSES, u_hat), 2)
#> [1] 0.51

## the fix: add the school mean of SES as a regressor (Mundlak 1978)
m_wb <- lmer(MathAch ~ SES + meanSES + Sector + (1 | School), data = hsb, REML = FALSE)
round(summary(m_wb)$coefficients, 3)
#>
#> Estimate Std. Error t value
#> (Intercept) 12.128 0.197 61.471
#> SES 2.191 0.109 20.164
#> meanSES 3.144 0.381 8.249
#> SectorCatholic 1.225 0.303 4.042
c(SSES_fixed_effects = coef(m_fe)["SES"])
#> SSES_fixed_effects.SES
#> 2.191
```

The SES coefficient is now the fixed-effects slope exactly: within a school, a unit of SES adds 2.19 points. The school-mean coefficient is the between-school part: a school whose students average one unit higher SES scores 3.14 points higher still, a contextual effect. Sector remains estimable. This is the random-intercept model with the fixed-effects assumption dropped.

8.4 A random-intercept logit

```
hsb$top <- as.integer(hsb$MathAch > quantile(hsb$MathAch, 0.75))
g1 <- glmer(top ~ SES + Sector + (1 | School), data = hsb,
            family = binomial, nAGQ = 10) # nAGQ: accuracy of the integral over u_j
round(summary(g1)$coefficients, 3)
#>
#> Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -1.460 0.077 -18.858 0
#> SES 0.674 0.045 14.979 0
#> SectorCatholic 0.391 0.112 3.494 0
as.data.frame(VarCorr(g1))$vcov
#> [1] 0.3386
```

The same logit as this morning, with a school shift on the log-odds scale whose variance is estimated. A coefficient is the log-odds change holding the school's u_j fixed; for a population-average probability, average predictions over the distribution of u_j .

9 Event history

The GSS extract has no duration variable, so this section simulates one. The advantage is that the truth is known, so each method can be checked against it.

9.1 A duration with censoring, by construction

Time, in years after age 18, to first union formation. College graduates form unions at a lower rate, a true log hazard ratio of -0.5 , and everyone is observed for at most 12 years, so long durations are censored.

```
set.seed(4)
n_eh <- 3000
sim <- tibble::tibble(
  college = rbinom(n_eh, 1, 0.35),
  rate     = exp(-2.2 - 0.5 * college), # hazard, events per year
  t_true   = rexp(n_eh, rate),          # the duration, never fully seen
  time     = pmin(t_true, 12),          # observed: censored at 12 years
  event    = as.integer(t_true <= 12)   # 1 = union observed, 0 = censored
)
sim |> count(college, event)
#> # A tibble: 4 x 3
#>   college event     n
#>   <int> <int> <int>
#> 1     0     0  511
#> 2     0     1 1462
#> 3     1     0  450
#> 4     1     1  577
```

A third to a half of each group is censored. That is the normal situation in event-history data, not a pathology.

9.2 Why the naive fixes fail

Before doing it right, do it wrong twice:

```
sim |>
  group_by(college) |>
  summarise(
    drop_censored = mean(time[event == 1]), # naive 1: drop censored
    censored_as_event = mean(time),          # naive 2: censoring = event
```

```

      truth          = mean(1 / rate)          # true expected duration
    ) |> round(2)
#> # A tibble: 2 x 4
#>   college drop_censored censored_as_event truth
#>   <dbl>      <dbl>          <dbl> <dbl>
#> 1     0         4.8            6.66  9.03
#> 2     1         5.35            8.26 14.9

```

Both naive columns understate the true durations, and they understate them *more for college graduates*, whose long durations are censored more often. The college gap itself is distorted, not just the level.

9.3 Kaplan-Meier

```

km <- survfit(Surv(time, event) ~ college, data = sim)
plot(km, col = c("navy", "firebrick"), lwd = 2,
     xlab = "Years since age 18", ylab = "Share still unpartnered")
legend("bottomleft", c("No college", "College"),
     col = c("navy", "firebrick"), lwd = 2, bty = "n")

```

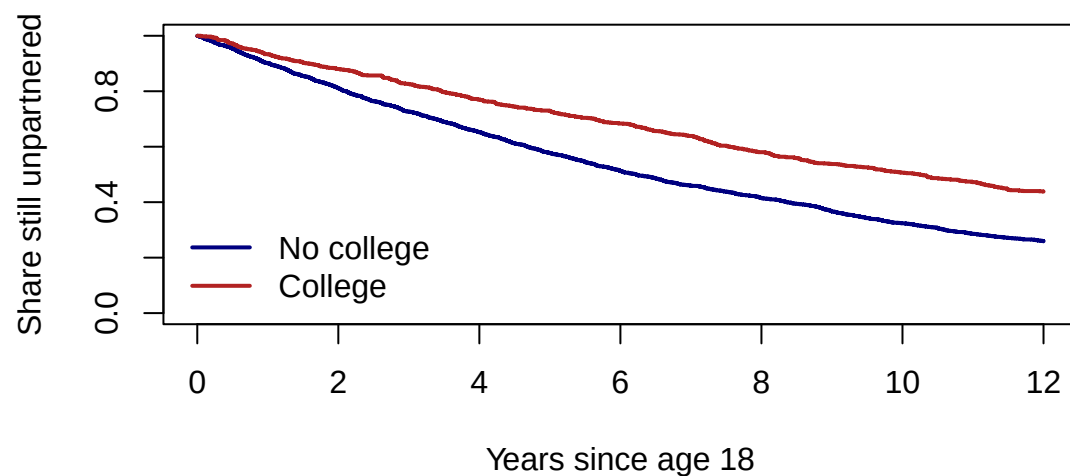


Figure 4: Kaplan-Meier survival curves by college. Each downward step is an observed union; censored units leave the risk set without a step.

9.4 The Cox model recovers the truth

```
fit_cox <- coxph(Surv(time, event) ~ college, data = sim)
summary(fit_cox)$coefficients |> round(3)
#>      coef exp(coef) se(coef)      z Pr(>|z|)
#> college -0.497    0.608    0.049 -10.09      0
```

The coefficient is the true -0.5 up to sampling noise, and $e^{\hat{\beta}} \approx 0.61$: college graduates form unions at about 61% of the non-college rate *at every point in time*. Censoring was handled by the likelihood, not by us.

9.5 Discrete time: the person-period logit

Cut time into years, give every person one row per year at risk, and run the logit from this morning:

```
## one row per person per year at risk; `period` indexes the year
pp <- survSplit(Surv(time, event) ~ college, data = sim,
               cut = 1:11, episode = "period")
c(rows_people = n_eh, rows_person_periods = nrow(pp))
#>      rows_people rows_person_periods
#>           3000           22661

## period dummies = a nonparametric baseline hazard
fit_dt <- glm(event ~ college + factor(period),
              family = binomial, data = pp)
round(coef(fit_dt)["college"], 3)
#> college
#>  -0.523
```

The person-period logit coefficient is essentially the Cox estimate: with small per-period hazards, the discrete-time logit and the proportional-hazards model agree. The period dummies play the role of the baseline hazard, and censored people simply stop contributing rows: the likelihood logic from lecture, implemented by data construction.

i Note

Everything from earlier in this lab now applies to `fit_dt`: robust standard errors (cluster on the *person*, since one person contributes many rows), average marginal effects, predicted per-period probabilities. That is the appeal of the discrete-time approach for sociologists.

10 Exercises

1. **Logit versus LPM for a causal contrast.** Estimate the “effect” of being ever-married (`evermar`) on holding a BA, controlling for `pareduc`, `wordsum`, `female`, `black`. Compare

the LPM coefficient with the logit AME. How close are they? Which would you report, and to whom?

2. **The rescaling problem.** Fit the logit for `ba` with (a) `pareduc` only and (b) `pareduc + wordsum + female + black`. Compare the `pareduc` *coefficient* across the two, then compare the `pareduc` *AME*. Explain the difference using Mood (2010).
3. **Simulation-based intervals.** Take the logit model and get an interval for the difference in predicted probability between `pareduc = 12` and `pareduc = 16` by simulating 10,000 draws of $\tilde{\beta}$ from $\mathcal{N}(\hat{\beta}, \hat{V}(\hat{\beta}))$ (King, Tomz, and Wittenberg 2000). Compare to the delta-method interval from `avg_comparisons()`.
4. **Offsets.** Suppose you wanted to model children *per year since age 18* rather than children. Add `offset(log(pmax(age - 18, 1)))` to the Poisson and interpret the coefficients. What changed?
5. **Hurdle versus zero-inflated.** Fit `pscl::zeroinfl(childs ~ educ + female + black + exper + evermar, dist = "negbin")` and add it to the AIC/BIC table. The two are not nested: which does each criterion prefer, and by how much? Then say which story fits the outcome: is a childless respondent someone who has not crossed a hurdle, or a different kind of person who could never have a positive count?
6. **A block test in the multinomial.** Drop `educ` from the region model and test it with an LR test (how many degrees of freedom?) and with BIC. Do the two agree? Explain in one sentence why they might not at $n = 3509$.
7. **Random slopes.** Extend the random-intercept model to `MathAch ~ SES + Sector + (1 + SES | School)`, so that the SES slope varies across schools. Report the variance of the slope and its correlation with the intercept, test the extension against the random-intercept model with an LR test, and say in one sentence what a school with a steep SES slope looks like.
8. **Censoring harder.** Rerun the event-history simulation with observation ending at 6 years instead of 12. How much censoring do you get now? Do Kaplan-Meier and the Cox model still recover the truth? What happens to the two naive estimates? The likelihood methods degrade gracefully; the naive ones get *worse* as censoring grows.

11 Session information

```
sessionInfo()
#> R version 4.5.1 (2025-06-13)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.6.2
#>
#> Matrix products: default
```

```

#> BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] pROC_1.19.0.1      brglm2_1.1.0      pscl_1.5.9
#> [4] nnet_7.3-20        AER_1.2-17        survival_3.8-3
#> [7] car_3.1-3          carData_3.0-5     MASS_7.3-65
#> [10] lmtest_0.9-40      zoo_1.8-14        sandwich_3.1-1
#> [13] marginaleffects_0.32.0 ggplot2_4.0.0     dplyr_1.1.4
#> [16] lme4_1.1-37        Matrix_1.7-3
#>
#> loaded via a namespace (and not attached):
#> [1] generics_0.1.4      enrichwith_0.5.0   lattice_0.22-7
#> [4] digest_0.6.37       magrittr_2.0.3     nleqslv_3.3.7
#> [7] evaluate_1.0.5      grid_4.5.1         RColorBrewer_1.1-3
#> [10] fastmap_1.2.0       jsonlite_2.0.0     backports_1.5.0
#> [13] Formula_1.2-5       scales_1.4.0       numDeriv_2016.8-1.1
#> [16] abind_1.4-8         reformulas_0.4.1   Rdpack_2.6.4
#> [19] cli_3.6.5           rlang_1.1.6        rbibutils_2.3
#> [22] splines_4.5.1       withr_3.0.2        yaml_2.3.10
#> [25] tools_4.5.1         checkmate_2.3.3    nloptr_2.2.1
#> [28] minqa_1.2.8         boot_1.3-31        vctrs_0.6.5
#> [31] R6_2.6.1            lifecycle_1.0.4    insight_1.4.6
#> [34] pkgconfig_2.0.3     pillar_1.11.0      gtable_0.3.6
#> [37] glue_1.8.0          data.table_1.17.8  Rcpp_1.1.0
#> [40] statmod_1.5.2       xfun_0.53          tibble_3.3.0
#> [43] tidyselect_1.2.1    knitr_1.50         farver_2.1.2
#> [46] htmltools_0.5.8.1   nlme_3.1-168       labeling_0.4.3
#> [49] rmarkdown_2.29      compiler_4.5.1     S7_0.2.0

```